

Company / Employee Management System — Architecture

Final Edition (V1) — original design plus a completed review pass. No implementation code below; this is architecture and reasoning. See Section L for a summary of everything this edition changed.

0. The one architectural insight that shapes everything else

Interfaces in Java cannot hold instance fields. That single fact drives the biggest decision in this design: how to give `CEO`, `CTO`, and `Manager` shared "I manage people" behavior without duplicating a `List<Employee>` field in three separate classes, and without forcing `Developer` / `Intern` to carry a subordinates list they will never use.

The answer: introduce one extra abstract class, `ManagementEmployee`, sitting between `Employee` and `{CEO, CTO, Manager}`. It holds the shared state (the subordinates list) and implements the shared interface (`TeamLead`), which supplies the shared recursive behavior (default methods). This is not a class added "to look layered" — it exists purely to solve the field-duplication problem. `Developer` and `Intern` skip it entirely because they do not manage anyone.

Everything below builds on this.

A. Final class hierarchy

```
Employee (abstract)
├── ManagementEmployee (abstract, implements TeamLead)
│   ├── CEO (final)
│   ├── CTO (final)
│   └── Manager (final)
├── Developer (abstract)
│   ├── SeniorDeveloper (final)
│   └── JuniorDeveloper (final)
└── Intern (final)
```

Why this hierarchy and not the flat 7-sibling one from the original prompt:

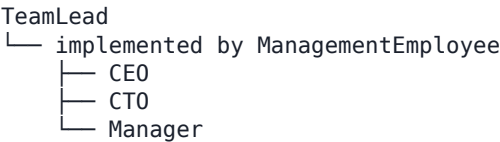
- `ManagementEmployee` is new — justified above (shared state + shared behavior for the three "has subordinates" roles).
- `Developer` stays abstract, not concrete-with-a-flag. `SeniorDeveloper` and `JuniorDeveloper` differ in more than a salary multiplier: a senior has a `List<JuniorDeveloper> mentees` and a `mentor(JuniorDeveloper)` method; a junior has a `SeniorDeveloper assignedMentor` reference and no one below it. That is a genuine behavioral/state difference, not a title difference — it earns the subclass split. If seniority only affected a salary number, this would collapse into one `Developer` class with a `seniorityLevel` field instead.
- `Intern` is a direct leaf off `Employee`, not off `Developer`, because interns are not necessarily technical (could be marketing/HR interns later) and are paid a stipend, not a salary — different enough state/behavior to not inherit from `Developer`.

- `Manager` is not a superclass of `CTO` / `CEO`. A CTO is not "a kind of Manager" semantically, and forcing that would give `CTO` a manager's fields it does not want. `ManagementEmployee` solves the same code-reuse problem without that false is-a relationship.

B. Interface relationships

Only one interface: `TeamLead`, implemented by `ManagementEmployee` (so `CEO/CTO/Manager` get it for free). Why only one, and why the other candidates got rejected:

Candidate interface	Verdict	Reasoning
<code>TeamLead</code> (add/remove/get subordinates, recursive team payroll, recursive report count, print team)	Kept	A genuine *sometimes* capability — only 3 of 7 employee types have it. That is exactly what an interface is for: a capability some-but-not-all implementers share, cutting across the inheritance tree.
<code>Payable</code> (<code>calculateSalary()</code>)	Rejected — kept as an abstract method directly on <code>Employee</code> instead	Every employee is payable — it is not a subset capability, it is core to what "Employee" means. Making it an interface would just be <code>Employee implements Payable</code> with no other implementer ever existing.
<code>Searchable</code> (as an interface)	Rejected — handled via <code>@Searchable</code> annotation + reflection instead (see §G)	You explicitly want annotation/reflection practice. Building both an interface-based search and an annotation-based one teaches the same lesson twice. Annotation-driven search also lets the search mechanism discover fields generically instead of every class writing its own <code>matches(String)</code> override.
<code>ReportGenerator</code> / display behavior	Rejected — a concrete <code>displayInfo()</code> method on <code>Employee</code> , overridable per subclass	Like <code>Payable</code> , this is universal, not a subset capability. No interface needed.



C. Class responsibility table

Updated in this edition: `Employee`'s manager-mutator is renamed and made `public` (was package-private — see the fix in §L.1); `ManagementEmployee` gains `getValidSubordinateTypes()`; every concrete employee type has a settled salary formula; `EmployeeManager` now has an explicit termination policy and computes `byType()` on

demand instead of storing a second map. The Relationships column is written out in full below (it was clipped by a narrow column in the earlier PDF export — not missing content, just an export artifact).

Class	Responsibility	Key fields	Key methods	Relationships
Employee	Common employee identity, state, and contract	<code>employeeId</code> (final), <code>name</code> , <code>department</code> , <code>employmentStatus</code> , <code>baseSalary</code> , <code>dateHired</code> (final), <code>manager</code> (nullable ref)	<code>calculateSalary()</code> (abstract); <code>displayInfo()</code> (concrete); <code>getEmployeeType()</code> (abstract); <code>setManagerInternal(Employee)</code> — public, internal-use-only, called only by <code>EmployeeManager</code> (see §H, §L.1); <code>equals()/hashCode()</code> (by <code>employeeId</code> only); <code>toString()</code> (concise debug form, distinct from <code>displayInfo()</code>)	Holds a reference to its own manager (<code>Employee</code> , nullable, parent pointer); tagged with a nested <code>EmployeeType</code> enum constant; superclass of every concrete employee type.
ManagementEmployee	Shared state/behavior for anyone who leads a team	<code>directReports</code> (final reference, mutable list)	Implements all <code>TeamLead</code> contract methods concretely; abstract <code>getValidSubordinateTypes()</code> — declares which <code>EmployeeType</code> values this role may legally manage (see §L.3)	Has-a list of direct-report <code>Employees</code> (child pointers); implements <code>TeamLead</code> ; parent class of <code>CEO</code> , <code>CTO</code> , <code>Manager</code> .
CEO	Top of hierarchy; no manager; company-wide bonus	<code>performanceBonusRate</code>	Overrides <code>calculateSalary()</code> → <code>baseSalary + baseSalary * performanceBonusRate</code> ; <code>getValidSubordinateTypes()</code> → <code>{CTO}</code>	Manages CTOs; has no manager of its own (always null); one active CEO per company is enforced by <code>Company</code> , not by this class (see §J).
CTO	Leads CTO's Managers; tech leadership bonus	<code>techLeadershipBonus</code>	Overrides <code>calculateSalary()</code> → <code>baseSalary + techLeadershipBonus</code> (flat); <code>getValidSubordinateTypes()</code> → <code>{MANAGER}</code>	Reports to the CEO; manages Managers.
Manager	Leads Developers/Interns; team-size bonus	<code>teamBonusRate</code>	Overrides <code>calculateSalary()</code> → <code>baseSalary + directReports.size() * teamBonusRate</code> ; <code>getValidSubordinateTypes()</code> → <code>{SENIOR_DEVELOPER, JUNIOR_DEVELOPER, INTERN}</code>	Reports to a CTO; manages <code>SeniorDevelopers</code> , <code>JuniorDevelopers</code> , and <code>Interns</code> . The only salary formula that actually needs <code>directReports</code> , which is a concrete reason this class lives under <code>ManagementEmployee</code> at all.
Developer	Shared technical-employee state	<code>programmingLanguage</code> , <code>projectsCompleted</code>	<code>calculateSalary()</code> stays abstract — pushed down further since senior/junior formulas genuinely differ	Reports to a Manager; superclass of <code>SeniorDeveloper</code> , <code>JuniorDeveloper</code> .

Class	Responsibility	Key fields	Key methods	Relationships
SeniorDeveloper	Mentors juniors	mentees (List<JuniorDeveloper>), seniorityMultiplier	mentor(JuniorDeveloper); overrides calculateSalary() → baseSalary * seniorityMultiplier	Has a separate, parallel mentoring relationship with JuniorDeveloper — not part of the manager/directReports tree at all.
JuniorDeveloper	Learning role	assignedMentor (SeniorDeveloper, nullable)	Overrides calculateSalary() → returns baseSalary as-is	Optionally paired with a SeniorDeveloper mentor, outside the reporting hierarchy.
Intern	Non-permanent, stipend-based	internshipEndDate, university, stipend	Overrides calculateSalary() → returns stipend, not a formula off baseSalary	Reports to a Manager; no subordinates of its own; not necessarily technical.
Company	Company identity + owns the CEO relationship	companyName (final), foundingDate (final), ceo (nullable)	hireAsCeo(CEO) — enforces "one active CEO" (see §J); delegates everything else to EmployeeManager	Has-a EmployeeManager (composition); has-a CEO (zero or one).
EmployeeManager	All cross-employee operations: add/remove/update/find/search/validate/assign	employeesById (final Map<String, Employee>)	hire(Employee); terminate(String id) — refuses (throws IllegalStateException) if the target still has direct reports, forcing an explicit reassignment first (see §L.5); findById(String); search(String query); assignManager(Employee, ManagementEmployee); companyWidePayroll(); byType() — builds Map<EmployeeType, List<Employee>> fresh from employeesById on every call (see §E, §L.6)	Owns and is the sole legitimate mutator of every cross-employee collection and relationship; the only class that calls Employee.setManagerInternal(...).
EmployeeIdGenerator	Generates unique IDs	counter (static final AtomicInteger)	static String nextId()	Static utility, no instances; consumed by EmployeeManager.hire(...).
TeamLead (interface)	Contract + reusable recursive algorithms for anyone managing people	—	Abstract: addSubordinate, removeSubordinate, getDirectReports. Default (implemented once, recursively, in terms of the abstract methods): countAllReports(), calculateTeamPayroll(), printTeam()	Implemented by ManagementEmployee; inherited by CEO, CTO, Manager.

D. static / final decision table

Field / method	Owner	Instance vs static	Final?	Why
<code>employeeId</code>	<code>Employee</code>	instance	final	Identity, set once at construction, never changes.
<code>dateHired</code>	<code>Employee</code>	instance	final	Does not change after the fact in V1 (a correction is a rare enough edge case to handle later if it ever comes up).
<code>name</code> , <code>department</code> , <code>employmentStatus</code> , <code>baseSalary</code>	<code>Employee</code>	instance	non-final	Legitimately changeable over an employee's tenure (raises, transfers, name changes).
<code>manager</code>	<code>Employee</code>	instance	non-final	Reassignable — but only through <code>EmployeeManager.assignManager(...)</code> , never a public setter called directly (see §H).
<code>directReports</code>	<code>ManagementEmployee</code>	instance	final reference, mutable contents	<code>private final List<Employee> directReports = new ArrayList<>();</code> — the list object itself never changes, but what is inside it does. "final ≠ immutable," worth sitting with.
<code>employeesById</code>	<code>EmployeeManager</code>	instance	final reference, mutable contents	Same pattern as above. (<code>employeesByType</code> is no longer a stored field in this edition — see §E, §L.6 — so it needs no static/final entry of its own; <code>byType()</code> is a plain method.)
<code>counter</code> (ID generator)	<code>EmployeeIdGenerator</code>	static	static final	Belongs to the mechanism, not to any one <code>Employee</code> — there is exactly one ID sequence for the whole application. Using <code>AtomicInteger</code> (not plain <code>int</code>) means the reference is <code>static final</code> even though <code>.incrementAndGet()</code> mutates its internal value — and it quietly makes this class thread-safe already, which pays off directly in V3.
Company-wide constants (e.g. <code>MAX_INTERN_MONTHS</code> , <code>DEFAULT_CURRENCY</code>)	<code>Company</code> or a small <code>Constants</code> holder	static	static final	Fixed rules of the domain, not tied to any instance.
<code>companyName</code> , <code>foundingDate</code>	<code>Company</code>	instance	final	Each <code>Company</code> object has its own identity — must not be static, or every company in the app would share one name (breaks multi-company support, see §J).
<code>ceo</code>	<code>Company</code>	instance	non-final	Mutable on purpose — CEOs change over time — but only through a validating method, never a bare setter.
Employee count (nowhere, deliberately)	—	—	—	<code>employeesById.size()</code> already gives you the count — a separate counter field would just be a second source of truth that can drift out of sync. This is the "static because it is convenient" trap to watch for — and the same reasoning that led to computing <code>byType()</code> on demand (§L.6).

E. Collections + generics architecture

The original `ArrayList<ArrayList<Employee>>` idea is rejected, and here is the specific failure mode: it works only if everyone remembers "index 2 means Manager," which is a magic number with no compiler support. Add an 8th employee type later and you either shift every existing index or bolt one onto the end and break the mental map. It also cannot answer "what type is this sub-list" at runtime without external documentation.

What is used instead:

- `Map<String, Employee> employeesById` — the primary store. $O(1)$ lookup by ID, the operation done constantly (find, remove, both sides of assign-manager).
- `Map<EmployeeType, List<Employee>> byType()` — a computed view, not a stored field. Built fresh from `employeesById` by `EmployeeManager.byType()` every time it is called. ~~rather than maintained as a second map kept in sync on every hire/terminate.~~

Why on-demand instead of a maintained second map: `employeesByType` as a stored, separately-updated field is a second source of truth relative to `employeesById` — exactly the same "can drift out of sync" risk that already ruled out a standalone employee-count field in §D. At this project's scale (a handful to a few hundred employees), walking `employeesById.values()` and bucketing by `getEmployeeType()` fresh on each call costs nothing measurable and removes the sync-bug class entirely — while still giving the exact same `Map<EmployeeType, List<Employee>>` generics practice. (If deliberately practicing keeping two structures consistent is preferred instead, the eager/maintained version is defensible too — just worth choosing on purpose rather than by default.)

`EmployeeType` is an enum with one constant per concrete subclass (`CEO`, `CTO`, `MANAGER`, `SENIOR_DEVELOPER`, `JUNIOR_DEVELOPER`, `INTERN`), and every `Employee` returns its own via an abstract `getEmployeeType()`. (An alternative uses `Class<? extends Employee>` itself as the map key via `getClass()`, skipping the enum entirely — worth knowing it exists, but the enum version is used deliberately for enum practice, and it does not drift the way a `Class`-key map's reflection subtleties can.)

Where generics show up beyond `List<Employee>`:

- A small `EmployeeUtils` class with a genuinely bounded generic method: `static <T extends Employee> double sumSalaries(List<T> employees)` — works on `List<CEO>`, `List<Developer>`, or `List<Employee>` alike.
- Wildcard practice, naturally motivated, not forced: `? extends Employee` (producer, read-only) for a display/report method like `printAll(List<? extends Employee> employees)`, since it only reads from the list. `? super` (consumer, write-only) for something like `collectAllSeniors(List<? super SeniorDeveloper> destination)`, where the caller might hand a `List<Employee>` or `List<Developer>` to dump results into — this is the one place `? super` shows up naturally rather than being shoehorned in.

No generic class (e.g. a generic `EmployeeManager<T>`) is recommended — there is no real second use case for it here, and forcing one would be over-engineering. Generic methods on an otherwise concrete class give the real practice without the artificial class.

F. Exception architecture

Four custom exceptions, deliberately not more — the checked-vs-unchecked distinction is the actual lesson here:

Exception	Checked / Unchecked	Thrown from	Rationale
<code>EmployeeNotFoundExcep tion</code>	Checked (extends <code>Exception</code>)	<code>EmployeeManager.findById</code> , <code>.terminate</code> , <code>.assignManager</code>	"ID does not exist" is a recoverable business condition the caller is expected to explicitly handle (e.g. show "not found" in a UI) — exactly what checked exceptions are for.

▼
processed by: `EmployeeManager.search(String query)`
|
▼
feature: fuzzy/partial text search across employees without `EmployeeManager`
needing to know each subclass's fields in advance

How the search actually works, conceptually: `EmployeeManager.search(query)` iterates every employee in `employeesById`, and for each one walks its runtime class's fields via reflection (`getClass().getDeclaredFields()`), checking `field.isAnnotationPresent(Searchable.class)`. One subtlety worth keeping: `getDeclaredFields()` only returns fields declared directly on that class, not inherited ones — so to catch `@Searchable` fields from both `Employee` and, say, `Developer`, the search method needs to walk up the superclass chain (`clazz.getSuperclass()`) collecting fields at each level, not just call `getDeclaredFields()` once. That is a real reflection lesson worth keeping, not flattening away.

For each `@Searchable` field found: `field.setAccessible(true)` (since names/department are private), `field.get(employee)`, convert to `String`, do a case-insensitive `contains()` against the query.

A second annotation (`@DisplayField`, driving a generic `printProfile()` the same way) is a natural, low-risk extension once `@Searchable` is working — treat it as optional polish, not core.

`@CompanyAction` (method-level, marking "company operations") is explicitly rejected — there is no consumer for it that adds real value here (an audit log or event system would be needed to make reflecting over method annotations worth it), and building one just to have somewhere for the annotation to "do something" is the annotation-framework trap to avoid.

H. Organizational hierarchy — object relationships and traversal

Both directions are stored, on purpose:

- Each `Employee` holds a `manager` reference (parent pointer) — walk up ("who does this person report to, and their manager, and so on").
- Each `ManagementEmployee` holds a `directReports` list (child pointers) — walk down.

The consistency problem, and how it is actually solved: if both sides were independently settable, nothing would stop `employee.setManager(newBoss)` from running without `newBoss.directReports` ever being updated — the tree would then lie depending which direction it is traversed from. The fix is access control, not cleverness: `Employee.manager` has no public setter at all. The only way to change a reporting relationship is `EmployeeManager.assignManager(Employee employee, ManagementEmployee newManager)`, which does both sides atomically — removes `employee` from the old manager's `directReports` (if any), adds it to the new manager's `directReports`, and calls `employee.setManagerInternal(newManager)` — all three steps, or none of them. (This becomes a real transaction in V2, and a real synchronization concern in V3 — noted for later.)

New in this edition — hierarchy validation: before doing the reassignment, `assignManager` checks `newManager.getValidSubordinateTypes().contains(employee.getEmployeeType())` and throws `InvalidHierarchyException` if it fails. This is what actually enforces "a Manager can only report to a CTO or CEO, never to another Manager" — see §C and §L.3.

New in this edition — termination with active reports: `EmployeeManager.terminate(id)` refuses (`IllegalStateException`) if `directReports` on the target is non-empty, forcing an explicit reassignment first. Auto-promoting orphaned reports up to the terminated employee's own manager is more interesting — and more of the recursive tree-surgery practice §9 of the original brief wants — but it is a sharper edge case (what happens when the terminated employee is the CEO, with nothing above to promote into?). Treated as a deliberate V1.1 extension once core CRUD is solid, not a day-one requirement — see §L.5.

Traversal happens through `TeamLead`'s default methods, written once against the interface's abstract contract and inherited everywhere: `printTeam()` (recursive, prints self then calls `printTeam()` on each direct report that is itself a `TeamLead`); `countAllReports()` (recursive: `directReports.size()` + the sum over direct reports that are `TeamLeads`); `calculateTeamPayroll()` (same recursive shape, summing `calculateSalary()`). Because these live as default methods on the interface, the recursion is written exactly once, and `CEO.printTeam()`, `CTO.printTeam()`, and `Manager.printTeam()` all just work.

```
Company
└─ ceo: CEO
    └─ directReports: [CTO, CTO, ...]
        └─ each CTO.directReports: [Manager, Manager, ...]
            └─ each Manager.directReports: [SeniorDeveloper | JuniorDeveloper | Intern, ...]
                (SeniorDeveloper.mentees is a SEPARATE, parallel relationship –
                 not part of the manager/directReports tree at all)
```

I. Package structure

<code>company/</code>	→ Company itself + any company-wide constants
<code>employee/</code>	→ Employee, ManagementEmployee, Developer (the abstract spine)
<code>employee/types/</code>	→ CEO, CTO, Manager, SeniorDeveloper, JuniorDeveloper, Intern (concrete leaves)
<code>enums/</code>	→ Department, EmploymentStatus (EmployeeType is nested inside Employee – see below)
<code>interfaces/</code>	→ TeamLead
<code>service/</code>	→ EmployeeManager, EmployeeIdGenerator
<code>exceptions/</code>	→ the 4 custom exception classes
<code>annotations/</code>	→ Searchable
<code>app/</code>	→ Main.java (the demo/entry point)

What stays out of each package, on purpose:

- `employee/` and `employee/types/` never contain cross-employee logic (no "find," no "validate uniqueness," no collection ownership) — that is exactly what `service/` exists to prevent from leaking into `Company`.
- `service/` never holds employee state beyond the collections it is currently managing — it orchestrates, it does not model.
- `exceptions/` and `annotations/` are intentionally thin, single-purpose packages — keeping them separate from `employee/` is what makes them easy to find later.

Confirms the fix in §L.1: `employee/` and `service/` stay separate packages, unchanged from the original design. This works because `Employee.setManagerInternal(...)` is `public` (not package-private) — package-private access in Java only works within the exact same package, so a package-private method on a class in `employee/` could never have been called from `EmployeeManager` in `service/` in the first place. The "internal use only" contract is enforced by naming and a Javadoc note, not the compiler, since Java has no narrower option for cross-package "only this one class may call this" access.

Inner / nested classes

One legitimate case, and nothing beyond it: `EmployeeType` as a nested enum inside `Employee` (`Employee.EmployeeType`, referenced as `Employee.EmployeeType.MANAGER`) — a real, common Java idiom (think `Thread.State`): a small enum whose entire purpose is classifying instances of its enclosing class belongs nested inside it, not floating in a generic `enums/` package next to unrelated types like `Department`. This is not invented for practice — it is the natural home for it.

`Department` and `EmploymentStatus` stay top-level in `enums/` — used across multiple unrelated classes, not tied to `Employee`'s identity the way `EmployeeType` is.

A Builder-pattern static nested class was considered (the original plan wanted Builder practice) and rejected here: `Employee`'s constructor does not have enough optional parameters to justify it (name, department, salary — that is it), and a Builder that has to account for inheritance (CEO's bonus fields vs. Intern's stipend) gets into self-referential generic builder territory fast — a lot of ceremony for a 3–4-argument constructor. See §L.8 for where Builder practice could land instead.

No non-static inner class anywhere — nothing in this design needs an object conceptually "owned by and entangled with a specific outer instance" the way a non-static inner class implies.

J. Singleton — evaluated, and rejected for CEO

Approach A (CEO as Singleton) is the wrong tool, and the precise reason is not just "it feels wrong": a classic Singleton guarantees one instance of a class in the entire JVM. "One CEO per company" is a completely different constraint — one active CEO per `Company` instance. These only look like the same rule if there is only ever one `Company` in the program.

The moment `Company A`, `Company B`, `Company C` are supported (which the V2/V3 roadmap implies is plausible), a CEO Singleton breaks immediately: it would force every company in the system to literally share the same CEO object, which is nonsensical.

Approach B — chosen: `Company` owns a `ceo` field and enforces the rule itself, at the point where a CEO is assigned. `hireAsCeo(CEO newCeo)` either replaces the outgoing CEO (a real-world "CEO transition") or throws if stricter "must fire the old one first" semantics are wanted — either way, the rule lives in `Company`'s business logic, not in `CEO`'s constructor or instantiation mechanism.

Where a Singleton would have been the right call, for contrast — honestly, nowhere. The closest candidate is `EmployeeIdGenerator`, since exactly one ID sequence is wanted for the whole app. But a formal Singleton (private constructor + `getInstance()` + held instance) is more machinery than this needs — there is no meaningful instance here beyond one counter. A plain class with a private constructor and static methods (a "static utility class," not a Singleton object) gets the same "exactly one ID sequence" guarantee with less ceremony. This project genuinely does not need a true Singleton anywhere, and that is a legitimate conclusion, not a gap: recognizing the difference between "only one of these should exist" (a business rule, enforced by validation) and "only one instance of this class should exist" (an actual Singleton) is the real lesson.

Enums — why these, not Strings

Enum	Values (example)	Why not a String
<code>Department</code>	<code>ENGINEERING, PRODUCT, SALES, HR, EXECUTIVE</code>	A <code>String department = "Egngineering"</code> typo compiles fine and fails silently at runtime. The enum makes invalid departments a compile error, and enables an exhaustive <code>switch</code> later.
<code>EmploymentStatus</code>	<code>ACTIVE, ON_LEAVE, TERMINATED</code>	Small, fixed set of states with meaningful transitions — exactly the enum use case, versus a boolean <code>isActive</code> that cannot represent "on leave" at all.

Enum	Values (example)	Why not a String
<code>Employee.EmployeeType</code> (nested — see §I)	CEO, CTO, MANAGER, SENIOR_DEVELOPER, JUNIOR_DEVELOPER, INTERN	Doubles as the key type for <code>byType()</code> 's returned map, and as the tag each subclass reports via <code>getEmployeeType()</code> .

J-2. V1 → V2 → V3 roadmap

V1 (this document) — core Java

Everything above. In-memory storage via `EmployeeManager`'s `employeesById` map, with `byType()` computed on demand.

V2 — JDBC + PostgreSQL

- `employees` table: `id` PK, `name`, `department`, `employment_status`, `base_salary`, `manager_id` (self-referential FK), `employee_type` (discriminator column), `date_hired`, plus a few nullable type-specific columns (`performance_bonus_rate`, `stipend`, etc.) rather than a full separate table per subclass — single-table-with-discriminator is the right complexity level for a learning project.
- `manager_id` replaces the in-memory `directReports` list as the source of truth — "who reports to X" becomes `SELECT * FROM employees WHERE manager_id = ?` instead of a Java list. Tree traversal (`countAllReports`, `calculateTeamPayroll`) either becomes a recursive SQL query (Postgres supports `WITH RECURSIVE`) or stays as Java recursion over freshly-queried children — worth a discussion when this stage arrives.
- An `EmployeeRepository` interface gets introduced, with `JdbcEmployeeRepository` as its implementation. `EmployeeManager` starts depending on `EmployeeRepository` instead of holding the `Map` directly — this is composition/dependency-swap in action, and it is the natural moment to see why coding against an interface here made this transition possible without rewriting `EmployeeManager`'s business logic.
- CRUD via `PreparedStatement`; a "hire + assign manager" style operation wraps in a single transaction so there is never a half-written state.
- This is also where `equals()/hashCode()` (§L.7) start earning their keep — two `Employee` objects built from the same row across two different queries should be equal, and the default identity-based `equals()` will not give that without the override already put in place in this edition.

V3 — concurrency, using the same domain

- Shared mutable state to interrogate: `EmployeeManager`'s map, and each `ManagementEmployee.directReports` list.
- Concrete race conditions to actually produce: two threads calling `hire()` concurrently on a plain `HashMap` — lost updates or corruption; a thread iterating `directReports` for `printTeam()` while another thread is mid-`assignManager()` — `ConcurrentModificationException`.
- Fixes, and why each fits where it does: `ConcurrentHashMap` for `employeesById` (write-heavy-ish, needs atomic per-key ops); `CopyOnWriteArrayList` for `directReports` (read-heavy — lots of `printTeam()/payroll` traversal, rare writes — the classic COW use case); a `ReentrantLock` (or `synchronized` block) around the whole `assignManager()` operation specifically, because it is a multi-step mutation touching two employees' state that must appear atomic to any concurrent reader.
- Already safe and needs nothing: `employeeId` (final, set once, never mutated), the enum constants themselves (inherently immutable), and — a bonus payoff from a V1 decision — `EmployeeIdGenerator`'s `AtomicInteger` counter, already thread-safe from the day it was written.

K. Final architecture diagram



Intern (final, direct child of Employee, stipend not salary formula)

@Searchable —placed on→ Employee.name, Employee.department, Developer.programmingLanguage
—processed by→ EmployeeManager.search(query), via reflection walking the
class + superclass field chain

L. Decisions finalized in this review pass

Everything above already reflects these; this section is a changelog against the pre-review draft, kept together so the reasoning is easy to find in one place.

#	Decision
L.1	<code>Employee.assignManager(Employee)</code> renamed to <code>setManagerInternal(Employee)</code> and made <code>public</code> (was package-private, which cannot compile across the <code>employee/</code> vs <code>service/</code> package boundary — Java package-private access only works within the exact same package). Intent is enforced by naming + Javadoc, not the compiler, since Java has no narrower cross-package access option. See §C, §I.
L.2	CEO's <code>manager</code> field stays <code>null</code> always — no board-of-directors concept needed for V1.
L.3	The Manager/CTO/CEO hierarchy rule is now enforced, not just discussed: <code>ManagementEmployee</code> declares abstract <code>getValidSubordinateTypes()</code> ; <code>CEO</code> → <code>{CTO}</code> , <code>CTO</code> → <code>{MANAGER}</code> , <code>Manager</code> → <code>{SENIOR_DEVELOPER, JUNIOR_DEVELOPER, INTERN}</code> . <code>assignManager</code> checks membership and throws <code>InvalidHierarchyException</code> on a mismatch. See §C, §H.
L.4	Salary formulas fixed for all six concrete types — see §C for the full table.
L.5	<code>EmployeeManager.terminate(id)</code> now refuses (throws the standard <code>IllegalStateException</code> , unchecked — no new custom exception type) if the target still has direct reports, forcing an explicit reassignment first. Auto-promotion of orphaned reports up to the terminated employee's own manager is deferred to a V1.1 extension. See §C, §F, §H.
L.6	<code>employeesByType</code> is no longer a stored, separately-maintained field. <code>EmployeeManager.byType()</code> computes <code>Map<EmployeeType, List<Employee>></code> fresh from <code>employeesById</code> on every call, removing the "two maps can drift apart" risk — the same reasoning that already ruled out a standalone employee-count field. See §D, §E.
L.7	<code>Employee</code> gains <code>equals()/hashCode()</code> based solely on <code>employeeId</code> , and a <code>toString()</code> distinct from <code>displayInfo()</code> (concise debug form vs. user-facing report). Optional but recommended — becomes load-bearing once V2 reconstructs <code>Employee</code> objects from database rows, where two objects from the same row across two queries should compare equal. See §C, §J-2.
L.8	Builder pattern confirmed out of scope for V1 (the original plan wanted Builder practice, but <code>Employee</code> 's constructor is not complex enough to earn it). If real Builder practice is still wanted, the honest answer is it does not live in this project as designed — the plausible spot is a query-builder over the JDBC layer in V2, only if it earns its place once that stage is reached. See §I.

Ready to start on `Employee` and `EmployeeType` — that is still the right first pair to implement, since almost everything else depends on them.